Software Requirements Specification

Restaurant App MVP

Mobile Application Development — Assignment 2

Document Version: 2.0

Prepared by: Ayesha

Institution: Abasyn University

Semester: 4th

Updated from Assignment 1 to include an Express server, MongoDB, REST API endpoints, authentication, and shared data across devices.

1. Introduction

1.1 Purpose

This SRS defines requirements for Restaurant App MVP version 2.0. It describes the React Native app, Node.js/Express server, MongoDB database, and API communication for Assignment 2. It is intended for the student developer and course instructor.

1.2 Scope

Customers register/login, browse and search menu items, manage a cart, place dine-in or takeaway orders, reserve tables, and track orders. Restaurant Managers view orders/reservations and manage menu items, prices, availability, order status, and reservation status. The app uses a REST API with JSON; Express uses Mongoose to store data in MongoDB.

Problems from A1 and how A2 solves them

A1 problem	A2 solution
Price changes on one phone did not reliably update other phones.	Save manager changes to MongoDB and fetch shared menu data through APIs.
Orders/reservations were local and not shared across devices.	Store records in MongoDB and retrieve them after login from any device.
Data could be lost after reinstalling the app.	Server-managed data remains in the database independently of the app installation.
Order status used a simulated timer.	Manager updates status through the API; tracking screen polls every 10 seconds.
Client could send stale totals or unavailable items.	Server checks availability and calculates totals from current database prices.
Two devices could reserve the same table/time.	Server checks capacity and overlapping bookings; conflicts return HTTP 409.

In scope: Express server, MongoDB, Mongoose models, seed data, JWT authentication, APIs for users/menu/orders/reservations, server validation, app API integration, and cross-device tests.

Out of scope: Real payments, payment-card storage, push notifications, delivery-driver tracking, and production hosting.

1.3 Definitions and Acronyms

Term	Definition
API	Interface used by app and server to communicate.
REST	API style using HTTP methods and resource URLs.
Endpoint	Specific method and URL, e.g. GET /api/menu.
JSON	Format used for API request/response data.
MongoDB / Collection	Document database / group of related documents.
Mongoose	Node.js library for MongoDB schemas and queries.
Middleware	Function in request processing, e.g. token verification.
JWT	Signed token used to identify an authenticated user.
HTTP Status Code	Number describing request result, e.g. 200, 401, 404.
SRS / UML / MVP	Requirements document / diagram language / minimum viable product.
AsyncStorage	Local key-value storage; in A2 used for session/token, not central business records.
FR / NFR	Functional Requirement / Non-Functional Requirement.

2. Overall Description

2.1 User Roles

Role	Permissions
Customer	Register/login; browse/search menu; manage cart; place orders; view own orders; reserve tables; view own reservations; track status; logout.
Restaurant Manager	View all orders/reservations; update valid order status; accept/decline reservations; add/edit/delete menu items and prices/availability. Protected actions require a valid JWT and manager role.

2.2 User Stories

• US-01: As a Customer, I want to register and log in so that I can use my own account.

• US-02: As a Customer, I want to browse categories and see images, descriptions, prices, and availability.

• US-03: As a Customer, I want to search by dish name while typing.

• US-04: As a Customer, I want to change cart quantities and notes before checkout.

• US-05: As a Customer, I want to apply a promo code and see the total.

• US-06: As a Customer, I want to book a table by date, time, and party size.

• US-07: As a Customer, I want to see my order status.

• US-08: As a Customer, I want orders/reservations to remain after reinstalling the app.

• US-09: As a Customer, I want an order on one device to appear on another after login.

• US-10: As a Manager, I want to see and update incoming orders.

• US-11: As a Manager, I want to accept or decline reservations.

• US-12: As a Manager, I want to update menu prices and availability for all customers.

• US-13: As a Manager, I want unauthorized changes to be rejected.

2.3 Customer Stories (Self-Interview)

• CS-01: “I choose food by pictures.” Show each item image, name, description, price, and availability.

• CS-02: “I want results while typing.” Provide case-insensitive search and a no-results message.

• CS-03: “I forget what is in my cart.” Show quantities, count, subtotal, discount, and total.

• CS-04: “I do not want to call to book.” Provide date/time selection, party size, and confirmation.

• CS-05: “I want to know when food is ready.” Show actual server status, not a fake timer.

• CS-06: “I do not want to lose my order history.” Store orders/reservations centrally.

2.4 Manager Stories

• MS-01: Update order status through Pending → Preparing → Ready → Served.

• MS-02: Accept/decline reservations to control seating.

• MS-03: Update menu items, prices, and availability for all customers.

• MS-04: Prevent customers from performing manager-only actions.

3. Functional Requirements

The following requirements preserve A1 features and update data operations to use the server.

3.1 Authentication

ID	Requirement
FR-01	The system shall allow registration with name, email, password, and role.
FR-02	The system shall validate email format and require at least 8 password characters including a digit.
FR-03	The system shall check that password and confirm-password fields match.
FR-04	The system shall hash passwords with bcrypt and never return password hashes.
FR-05	The system shall issue a one-day JWT after successful login.
FR-06	The system shall store token and safe user details in AuthContext and AsyncStorage and clear them on logout.
FR-07	The system shall return 401 for missing/invalid tokens and 403 for customer access to manager-only routes.

3.2 Menu and Search

ID	Requirement
FR-08	The system shall fetch menu items from GET /api/menu instead of mock files.
FR-09	The system shall show name, description, price, category, image, daily-special flag, and availability.
FR-10	The system shall filter by category and search case-insensitively with about 400 ms debounce.
FR-11	The system shall disable Add-to-Cart for unavailable items and show no-results state.
FR-12	The system shall show loading, error, Retry, and pull-to-refresh states.
FR-13	The system shall allow only managers to create, edit, or delete menu items.
FR-14	The system shall share manager price/availability changes through the server.
FR-15	The system shall keep the last five unique searches locally and return 400/404 for invalid/missing menu requests.

3.3 Cart and Orders

ID	Requirement
FR-16	The system shall allow customers to add/remove items and change quantities.
FR-17	The system shall allow a note for each item and promo codes WELCOME10 or FEAST20.
FR-18	The system shall show item count, subtotal, discount, and estimated total.
FR-19	The system shall support Dine-in with table selection or Takeaway with pickup time.
FR-20	The system shall save orders through POST /api/orders with user, items, quantities, type, status, and timestamps.
FR-21	The system shall calculate totals from current database prices and reject unavailable items.
FR-22	The system shall let customers retrieve their orders and managers retrieve all orders.
FR-23	The system shall allow status progression Pending → Preparing → Ready → Served; cancellation is allowed only while Pending.
FR-24	The system shall refresh status from the server every 10 seconds and clean up polling when the screen closes.
FR-25	The system shall remove the A1 fake timer as the source of order status.

3.4 Reservations

ID	Requirement
FR-26	The system shall prevent past dates and offer time slots from 12:00 to 22:00 at least one hour ahead.
FR-27	The system shall validate party size from 1 to 12 and Pakistani phone format 03XX-XXXXXXX.
FR-28	The system shall create reservations through POST /api/reservations and retrieve the customer's reservations.
FR-29	The system shall allow only managers to view all reservations and accept/decline them.
FR-30	The system shall check table capacity and overlapping bookings on the server; conflicts return 409.
FR-31	The system shall persist reservations and timestamps in MongoDB.

3.5 Manager Dashboard and API

ID	Requirement
FR-32	The system shall provide dashboard tabs for orders, reservations, and menu management.
FR-33	The system shall require JWT and manager role for protected management endpoints.
FR-34	The system shall reload relevant data after successful manager updates and show errors when requests fail.
FR-35	The system shall share central menu, order, and reservation data across devices.
FR-36	The system shall provide GET /api/health with status and current time.
FR-37	The system shall use JSON and appropriate HTTP status codes.
FR-38	The system shall return 400 invalid input, 401 unauthenticated, 403 forbidden, 404 missing resource/route, 409 conflict, and 500 unexpected error.
FR-39	The system shall use src/api/client.js for BASE_URL, JSON headers, token headers, and server errors.
FR-40	The system shall provide a useApi hook returning data, loading, error, and refetch, used in at least two screens.
FR-41	The system shall use the laptop LAN IP for a physical phone, not localhost.

4. Non-Functional Requirements

4.1 Usability

• The system shall use consistent layouts, labels, icons, and colours.

• The system shall show loading, success, empty, and error states and understandable messages.

4.2 Performance and Compatibility

• The app shall use FlatList virtualization for long lists and show loading during requests.

• The UI shall adapt to phone sizes and safe areas.

• A physical phone and laptop must be on a reachable network; local testing uses the laptop IP.

• Order polling shall stop when the tracking screen is closed.

4.3 Security

• Passwords shall be hashed with bcrypt and never returned by APIs.

• JWT_SECRET and MONGO_URI shall stay in .env; do not commit .env.

• Protected routes verify Bearer tokens; manager routes check role.

• Only .env.example with placeholder values shall be committed.

4.4 Reliability, Maintainability, and Persistence

• MongoDB shall be the source of truth for users, menu, tables, orders, and reservations.

• The server shall validate orders and reservation conflicts before saving.

• Server folders shall separate config, models, routes, controllers, and middleware.

• API requests shall live in src/api/client.js; reusable request state shall live in src/hooks/useApi.js.

• AsyncStorage shall store session data, not central business records.

5. Database Design

MongoDB collections replace A1 mock data. Mongoose schemas define types, validation, defaults, and references; each document has an _id.

Collection	Fields and types	Required / unique rules	Relationships and rules
Users	name String; email String; passwordHash String; role String; timestamps	Name/email/passwordHash/role required; email unique and lowercase; role customer/manager.	One user has many orders/reservations. Never store plain passwords.
MenuItems	name, description, category, image Strings; price Number; isSpecial/isAvailable Boolean	Name, price, category required; price min 0; special default false; available default true.	Order line refers to MenuItems._id and stores name/unitPrice snapshot.
Tables	tableNumber Number; capacity Number; isAvailable Boolean	tableNumber unique; capacity min 1; available default true.	Reservation.tableId references Tables._id; server checks booking conflicts.
Reservations	userId/tableId ObjectId; date; time; partySize; contactPhone; status; timestamps	Required user/table/date/time/partySize/phone; partySize min 1; status enum pending/accepted/declined/cancelled.	userId→Users; tableId→Tables. Capacity and overlap checked server-side; conflict 409.
Orders	userId ObjectId; items[{menuItem,name,unitPrice,quantity,note}]; total; type; tableId/pickupTime; status; timestamps	Required user/items/total/type/status; total min 0; quantity min 1; type dine-in/takeaway; status Pending/Preparing/Ready/Served/Cancelled.	userId→Users; items.menuItem→MenuItems; tableId→Tables for dine-in. Server recalculates total.

5.1 Data Rules

• Duplicate email returns 409; menu price cannot be negative and quantity must be at least one.

• Order status progresses Pending → Preparing → Ready → Served; cancellation is allowed only while Pending.

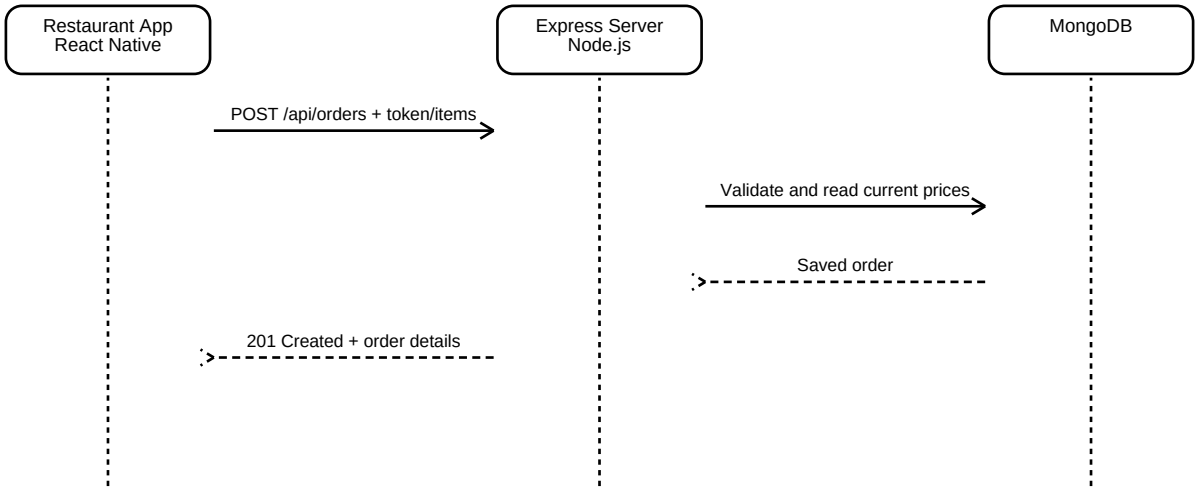
• Reservations must fit table capacity and not overlap an active booking for the same table.

• Orders and reservations have createdAt and updatedAt timestamps.

6. UML Diagrams

6.1 Sequence Diagram — Customer Places an Order

The App sends the order request; Express validates it, reads current prices and availability, asks MongoDB to save the order, and returns the result.



Server calculates total and checks availability before saving.

Flow: (1) POST /api/orders with Bearer token and requested items. (2) Server validates user, item IDs, quantities, and availability. (3) Server calculates total from current prices. (4) MongoDB saves the order. (5) Server returns 201 Created and the saved order; invalid requests receive an error.

6.2 Deployment Diagram

The phone runs the React Native app; the laptop runs Express on port 5000; the server connects to MongoDB Atlas or a local MongoDB instance.

Phone uses laptop LAN IP, not localhost; both devices use the same Wi-Fi.

7. Mobile App Screens

A1 screens and navigation remain. Server-managed data is fetched through APIs; network screens keep loading, error, retry, and refresh states.

Section	Purpose, UI, navigation, data, and hooks
7.1 Login / Signup	Register/login; name, email, password, role, submit/loading. APIs POST /api/auth/register and /api/auth/login. Save token/safe user in AuthContext and AsyncStorage. useState/auth hook.
7.2 Menu	Browse categories, item cards, image, price, availability, special badge, Add, refresh. GET /api/menu with optional category/search. useApi/useEffect and FlatList.
7.3 Search	Search box, clear, recent searches, no-results. Search current menu case-insensitively; debounce around 400 ms. useRef/debounce.
7.4 Cart	Quantities, notes, promo code, subtotal/discount/total, order type. POST /api/orders; server recalculates total. CartContext/useReducer.
7.5 Reservation	Date/time, party size, table, phone, confirmation, My Reservations. POST /api/reservations and GET /api/reservations/my; manager uses GET/PATCH reservations. useReservation/useApi.
7.6 Order Tracking	Order summary, status progress, loading/error. GET /api/orders/my every 10 seconds while active; remove fake timer. useEffect/setInterval with cleanup.
7.7 Profile	Show name/email/role, theme, logout. Logout clears context and AsyncStorage session. useAuth/useContext.
7.8 Manager Dashboard	Tabs for orders, reservations, menu; status controls, accept/decline, add/edit/delete items, price/availability. Protected APIs; reload after updates. useApi/useEffect.

8. API Endpoints

API requests and responses use JSON. Protected endpoints require Authorization: Bearer ; manager-only endpoints also require role=manager.

Method	Endpoint	Access	Request body / query	Response / purpose
GET	/api/health	Public	None	200 status and current server time
POST	/api/auth/register	Public	name, email, password, role	201 safe user; 409 duplicate email
POST	/api/auth/login	Public	email, password	200 token + safe user; 401 invalid credentials
GET	/api/menu	Public	Optional category, search	200 list of menu items
GET	/api/menu/:id	Public	ID in URL	200 item; 400 invalid ID; 404 missing
POST	/api/menu	Manager	name, description, price, category, image, isSpecial, isAvailable	201 created item; 400 invalid
PUT	/api/menu/:id	Manager	Fields to update	200 updated item; 400/404 errors
PATCH	/api/menu/:id/availability	Manager	isAvailable Boolean	200 updated availability
DELETE	/api/menu/:id	Manager	ID in URL	200 confirmation; 404 missing
POST	/api/orders	Customer	items[{menuItem,quantity,note}], type, tableId or pickupTime	201 saved order with server total
GET	/api/orders/my	Customer	Token	200 customer orders
GET	/api/orders	Manager	Token	200 all orders
PATCH	/api/orders/:id/status	Manager	status	200 updated order; 400 invalid progression
POST	/api/reservations	Customer	tableId, date, time, partySize, contactPhone	201 reservation; 409 time conflict
GET	/api/reservations/my	Customer	Token	200 customer reservations
GET	/api/reservations	Manager	Token	200 all reservations
PATCH	/api/reservations/:id	Manager	status accepted or declined	200 updated reservation; 404 missing

8.1 HTTP Status Codes

Code	Meaning
200 OK	Request succeeded.
201 Created	Resource created.
400 Bad Request	Invalid fields/ID, unavailable item, or invalid status transition.
401 Unauthorized	Missing/invalid token or login credentials.
403 Forbidden	Authenticated user lacks manager permission.
404 Not Found	Unknown route or resource.
409 Conflict	Duplicate email or conflicting reservation.
500 Internal Server Error	Unexpected server/database error returned as JSON.

8.2 API Client and Network Configuration

Use src/api/client.js as the only location for BASE_URL. The helper adds JSON headers, attaches the Bearer token when available, parses responses, and throws useful error messages. A physical phone must use the laptop LAN IP (for example http://192.168.1.15:5000), not localhost; both devices must use the same Wi-Fi. For Android emulator use http://10.0.2.2:5000.

8.3 Testing and Submission Notes

• Test GET /api/health in Postman and the phone browser.

• Test success and failure cases including missing name, negative price, invalid ID, missing token, customer token on manager route, and conflicting reservation.

• Use two devices to confirm manager price/availability changes appear after refresh and order/reservation changes appear without restarting the app.

• Keep .env private and commit only .env.example. Backend stays in server/config, models, routes, controllers, middleware; mobile code stays in src and API client in src/api.